UNIVERSIDAD DE GUADALAJARA

CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS
Departamento de Innovación Basada en la Información y el Conocimiento

Ingeniería En Computación

PRÁCTICA 6

Tareas Paralelas y Descomposición

Comparación de rendimiento entre suma secuencial y multiprocessing.Pool(4)

Nombre: GONZALEZ MARTINEZ ALONDRA GORETTI	**Código**: 218534681
Materia: Programación paralela y concurrente	**Grupo**: _____
Nombre del profesor: Mtro. Zeus Cobián	**Actividad**: Práctica 6 (100 puntos)
Fecha de entrega: 13 de noviembre del 2026	**Lenguaje**: Python 3.12 · multiprocessing

Bibliografía:
Python Software Foundation. (s. f.). *multiprocessing — Process-based parallelism*. Python 3 Documentation.
Python Software Foundation. (s. f.). *concurrent.futures — Launching parallel tasks*. Python 3 Documentation.
Mattson, T., Sanders, B. y Massingill, B. (2004). *Patterns for Parallel Programming*. Addison-Wesley.
Recursos didácticos de la asignatura (Semana 9 – Tareas paralelas y descomposición de problemas).

1. Objetivo de la práctica

Implementar la descomposición de un problema en tareas paralelas: sumar una lista de 100,000 números aleatorios entre 1 y 100 dividiéndola en 4 segmentos iguales que se procesan con el módulo `multiprocessing`, y comparar su tiempo con el de una versión secuencial para analizar cuándo el paralelismo mejora el rendimiento.

2. Fundamento breve

La **descomposición de problemas** consiste en dividir un problema complejo en subtareas lo más independientes posible, de modo que puedan ejecutarse simultáneamente. Puede hacerse por **datos** (*data parallelism*: el mismo cálculo sobre distintos fragmentos) o por **funciones** (*task parallelism*: cálculos distintos a la vez). La suma de una lista es un caso clásico de descomposición por datos: cada segmento se suma sin necesitar a los demás y al final se combinan los resultados parciales (esquema *dividir – procesar – combinar*).

En Python, los hilos comparten el GIL (*Global Interpreter Lock*), así que el cálculo intensivo de CPU solo se acelera con **procesos**, cada uno con su propio intérprete. A cambio, crear procesos y enviarles los datos (serialización con *pickle*) tiene un costo. Por eso el beneficio depende de que el trabajo por tarea sea suficientemente grande frente a esa sobrecarga y de que existan núcleos físicos disponibles (Python Software Foundation, s. f.).

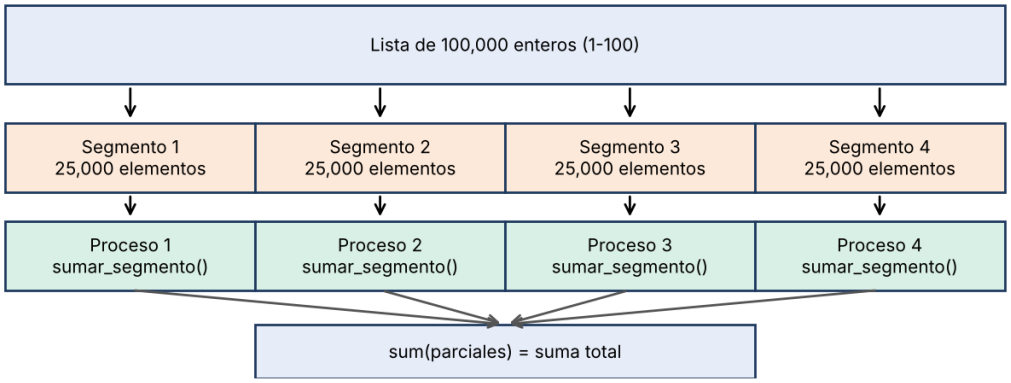


Figura 1. Descomposición de datos del problema. *Fuente: elaboración propia.*

Metodología de medición

- Lista de 100,000 enteros con `random.Random(semilla).randint(1, 100)`; la misma lista se usa en ambas versiones y se verifica que las dos sumas coincidan.
- Se mide con `time.perf_counter()` solo el cálculo: en la versión paralela se incluye dividir la lista, crear el `Pool(4)`, enviar los segmentos y sumar los parciales.
- Corrida de referencia con semilla 2025, 10 corridas con listas distintas y dos experimentos adicionales (tamaño de la lista y carga por elemento).
- Entorno de prueba: Python 3.12.3, Linux 6.18.44-fc-v242, **1 núcleo(s) lógico(s) disponible(s)**. Los tiempos absolutos dependen del equipo; la conclusión sobre núcleos disponibles es la clave para interpretarlos.

3. Código de la práctica

3.1 Programa completo (sumas.py)

```
"""Práctica 6 · Tareas paralelas y descomposición de problemas.

Suma de una lista de 100,000 números aleatorios (1-100):
  1) versión SECUENCIAL
  2) versión PARALELA con multiprocessing.Pool(4) (descomposición de datos)

Uso: python sumas.py            (semilla por defecto 2025)
     python sumas.py --semilla 7
"""
import argparse
import multiprocessing as mp
import os
import random
import time

N = 100_000
PROCESOS = 4


def generar_lista(n=N, semilla=None):
    """Lista de n enteros aleatorios entre 1 y 100."""
    rng = random.Random(semilla)
    return [rng.randint(1, 100) for _ in range(n)]


def sumar_segmento(segmento):
    """Tarea paralela: recibe un segmento de la lista y devuelve su suma."""
    total = 0
    for x in segmento:
        total += x
    return total


def dividir(lista, partes=PROCESOS):
    """Descomposición de datos: 'partes' segmentos contiguos del mismo tamaño."""
    tam = len(lista) // partes
    return [lista[i * tam:(i + 1) * tam] for i in range(partes)]


def suma_secuencial(lista):
    t0 = time.perf_counter()
    total = sumar_segmento(lista)              # una sola tarea con todos los datos
    return total, time.perf_counter() - t0


def suma_paralela(lista, procesos=PROCESOS, pool=None):
    """Si se pasa un pool ya creado, no se cuenta el costo de crearlo."""
    t0 = time.perf_counter()
    segmentos = dividir(lista, procesos)
    if pool is None:
        with mp.Pool(procesos) as p:           # crear el pool SÍ cuenta
            parciales = p.map(sumar_segmento, segmentos)
    else:
        parciales = pool.map(sumar_segmento, segmentos)
    total = sum(parciales)                     # combinar resultados parciales
    return total, parciales, time.perf_counter() - t0


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--semilla", type=int, default=2025)
    args = ap.parse_args()

    lista = generar_lista(semilla=args.semilla)
    print(f"Elementos: {len(lista):,} | Núcleos disponibles: {os.cpu_count()} | Procesos: {PROCESOS}")
    print(f"Tamaño de cada segmento: {len(lista) // PROCESOS:,}\n")

    print("=== BLOQUE 1: ejecución SECUENCIAL ===")
    t_sec_total, t_sec = suma_secuencial(lista)
    print(f"Suma total: {t_sec_total:,}")
    print(f"--> Tiempo secuencial: {t_sec:.6f} s\n")

```

```
73      print("=== BLOQUE 2: ejecución PARALELA (multiprocessing, 4 procesos) ===")
74      t_par_total, parciales, t_par = suma_paralela(lista)
75      for i, s in enumerate(parciales, 1):
76          print(f"Proceso {i}: suma parcial = {s:,}")
77      print(f"Suma total: {t_par_total:,}")
78      print(f"--> Tiempo paralelo: {t_par:.6f} s\n")
79
80      assert t_sec_total == t_par_total == sum(lista), "las sumas no coinciden"
81      print("Verificación: ambas sumas coinciden con sum(lista).\n")
82
83      print("=== TABLA COMPARATIVA ===")
84      print(f"{'Versión':<24}{'Tiempo (s)':>14}")
85      print(f"{'Secuencial':<24}{t_sec:>14.6f}")
86      print(f"{'Paralelo (4 procesos)':<24}{t_par:>14.6f}")
87      print(f"{'Aceleración':<24}{t_sec / t_par:>13.2f}x")
88
89
90  if __name__ == "__main__":
91      main()
```

Los scripts `experimento.py` y `generar_informe.py` (incluidos en la entrega) repiten las mediciones y generan las tablas y gráficas de la sección 4. Todo se ejecuta con `python sumas.py`, `python experimento.py` y `python generar_informe.py`.

4. Resultados

4.1 Captura de la ejecución

Salida real del programa con la semilla 2025:

```
$ python sumas.py --semilla 2025

Elementos: 100,000 | Núcleos disponibles: 1 | Procesos: 4

Tamaño de cada segmento: 25,000


=== BLOQUE 1: ejecución SECUENCIAL ===

Suma total: 5,054,170

--> Tiempo secuencial: 0.001680 s


=== BLOQUE 2: ejecución PARALELA (multiprocessing, 4 procesos) ===

Proceso 1: suma parcial = 1,266,049

Proceso 2: suma parcial = 1,264,992

Proceso 3: suma parcial = 1,262,346

Proceso 4: suma parcial = 1,260,783

Suma total: 5,054,170

--> Tiempo paralelo: 0.033194 s


Verificación: ambas sumas coinciden con sum(lista).


=== TABLA COMPARATIVA ===

Versión                    Tiempo (s)

Secuencial                   0.001680

Paralelo (4 procesos)        0.033194

Aceleración                     0.05x
```

Figura 2. Ejecución de la versión secuencial y de la paralela. *Fuente: elaboración propia.*

4.2 Comparación

Tabla 1. Tabla de comparación de la corrida de referencia (semilla 2025, 100,000 elementos).

Versión	Tiempo de ejecución (s)	Tiempo (ms)	Aceleración
Secuencial	0.001680	1.68	1.00×
Paralelo (4 procesos)	0.033194	33.19	0.05×

Tabla 2. Diez corridas con listas aleatorias distintas (100,000 elementos).

Corrida	Secuencial (ms)	Paralelo 4 procesos (ms)	Aceleración
1	1.58	21.97	0.07×
2	1.68	17.47	0.10×
3	1.65	14.22	0.12×
4	3.11	23.06	0.13×
5	1.60	16.82	0.10×
6	1.61	17.24	0.09×
7	1.67	14.51	0.12×
8	1.61	17.87	0.09×
9	1.62	15.72	0.10×
10	1.59	13.95	0.11×
Promedio	**1.77**	**17.28**	**0.10×**

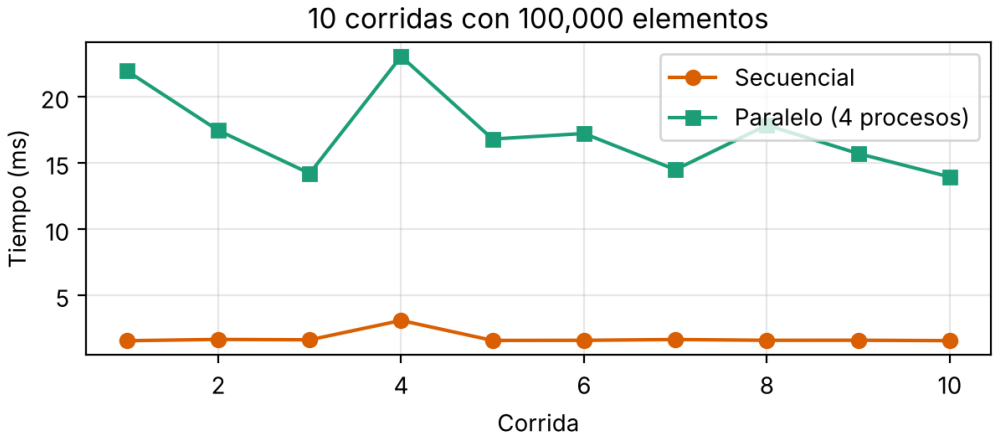


Figura 3. Tiempo de cada versión en las 10 corridas. *Fuente: elaboración propia.*

4.3 Efecto del tamaño de la lista y del costo de crear los procesos

Tabla 3. Mediana de varias repeticiones según el número de elementos. «Pool ya creado» excluye el costo de arrancar los procesos.

Elementos	Secuencial (ms)	Paralelo creando pool (ms)	Paralelo pool ya creado (ms)	Acel. (con pool)
10,000	0.17	13.92	1.17	0.01×
100,000	1.72	16.44	4.61	0.10×
1,000,000	16.94	62.59	42.19	0.27×
10,000,000	162.43	476.67	386.72	0.34×

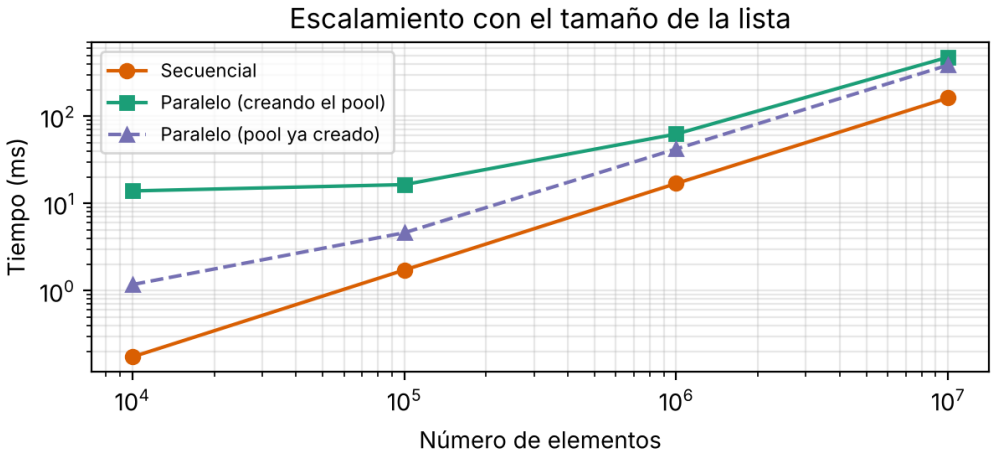


Figura 4. Escalamiento con el tamaño (ejes logarítmicos). *Fuente: elaboración propia.*

4.4 Carga con más cálculo por elemento

Para comprobar el efecto de la granularidad se repitió la comparación con 100,000 elementos, pero cada uno requiere 100 operaciones en lugar de una (`tarea_pesada` en `experimento.py`).

Tabla 4. Mediana de 3 repeticiones con carga pesada por elemento.

Versión	Tiempo (s)	Aceleración
Secuencial	0.423	1.00×
Paralelo (4 procesos)	0.338	1.25×

Nota: los tiempos varían de una ejecución a otra y entre equipos. Este informe se midió en un equipo con 1 núcleo(s) lógico(s); en una computadora con 4 o más núcleos la versión paralela con carga pesada debería acercarse a una aceleración mayor que 1× (ver la sección 5).

5. Reflexión final

¿Por qué el resultado se ve así y qué dice sobre la descomposición? En esta medición la versión **secuencial fue más rápida**: 1.77 ms frente a 17.28 ms en promedio (9.7 veces más lenta la paralela). No es una falla de la descomposición, sino de su costo: sumar 100,000 enteros toma apenas 1–2 ms, mientras que crear 4 procesos, serializar los segmentos con *pickle*, enviarlos y recolectar los resultados cuesta decenas de milisegundos. Además, el equipo de prueba solo ofrece 1 núcleo(s), de modo que los 4 procesos se turnan en el mismo núcleo y no hay ejecución simultánea real. La Tabla 3 muestra que al crecer la lista la brecha se reduce (el trabajo útil crece y el costo fijo de los procesos se vuelve relativo), y que sin el costo de crear el pool la versión paralela mejora notablemente. La Tabla 4 confirma el principio de *granularidad*: la paralelización solo conviene cuando cada tarea realiza suficiente cálculo.

¿Cuándo conviene esta técnica en programas reales? Cuando el problema se puede partir en subtareas independientes con mucho cálculo por tarea y hay núcleos disponibles: procesamiento de imágenes o video por bloques, simulaciones (Monte Carlo), entrenamiento y evaluación de modelos con validación cruzada, análisis de grandes archivos de datos por fragmentos y cálculo científico con matrices. Para esperas de red o disco es más apropiada la asincronía (Práctica 5); para cálculo intensivo de CPU, los procesos; y para tareas muy pequeñas, la versión secuencial.

Limitaciones. La ganancia máxima está acotada por la fracción secuencial del programa (ley de Amdahl): aquí dividir, enviar y combinar son partes no paralelizables. Con p núcleos reales la aceleración ideal es menor que p, nunca mayor.

6. Conclusión

Se implementó la descomposición por datos de la suma de 100,000 enteros en 4 segmentos con `multiprocessing.Pool(4)`; ambas versiones obtuvieron el mismo resultado (5,054,170 con la semilla 2025). En el equipo de prueba (1 núcleo) la versión **secuencial** fue la más rápida: 1.68 ms frente a 33.19 ms en la corrida de referencia y 1.77 ms frente a 17.28 ms en promedio de 10 corridas. La práctica muestra que paralelizar no siempre es más rápido: el beneficio depende de que cada tarea tenga trabajo suficiente para compensar la creación de procesos y la transferencia de datos, y de contar con núcleos físicos disponibles. La descomposición correcta (segmentos independientes, mismo tamaño y combinación final de parciales) es necesaria, pero no suficiente.

Referencias

Python Software Foundation. (s. f.). *multiprocessing — Process-based parallelism*. Python 3 Documentation. https://docs.python.org/3/library/multiprocessing.html

Python Software Foundation. (s. f.). *concurrent.futures — Launching parallel tasks*. Python 3 Documentation. https://docs.python.org/3/library/concurrent.futures.html

Mattson, T. G., Sanders, B. A. y Massingill, B. L. (2004). *Patterns for Parallel Programming*. Addison-Wesley.

Gutiérrez Cobián, Z. E. (2026). *Semana 9 – Tareas paralelas y descomposición de problemas* [Presentación]. CUCEI, Universidad de Guadalajara.